

IPD434

Transformers y Atención

Dr. Patricio Olivares Roncagliolo

¿Qué es el procesamiento del lenguaje natural?

El procesamiento del lenguaje natural se abrevia PLN en español y NLP en inglés. Es el área de la inteligencia artificial que permite a los computadores analizar, interpretar y generar lenguaje humano.

- Convierte texto o voz en representaciones que un modelo puede procesar.
- Aprende patrones a partir de ejemplos para producir predicciones o respuestas.
- Incluye tareas como clasificación de textos, traducción, búsqueda, resumen y diálogo.

Esta unidad presenta representaciones, atención y la arquitectura Transformer, que permiten resolver muchas de estas tareas.

Aprendizaje profundo para PLN: tokenización



- Tokenizar es dividir el texto en unidades que el modelo pueda procesar.
- A continuación veremos los métodos, la convención y las herramientas del esquema.

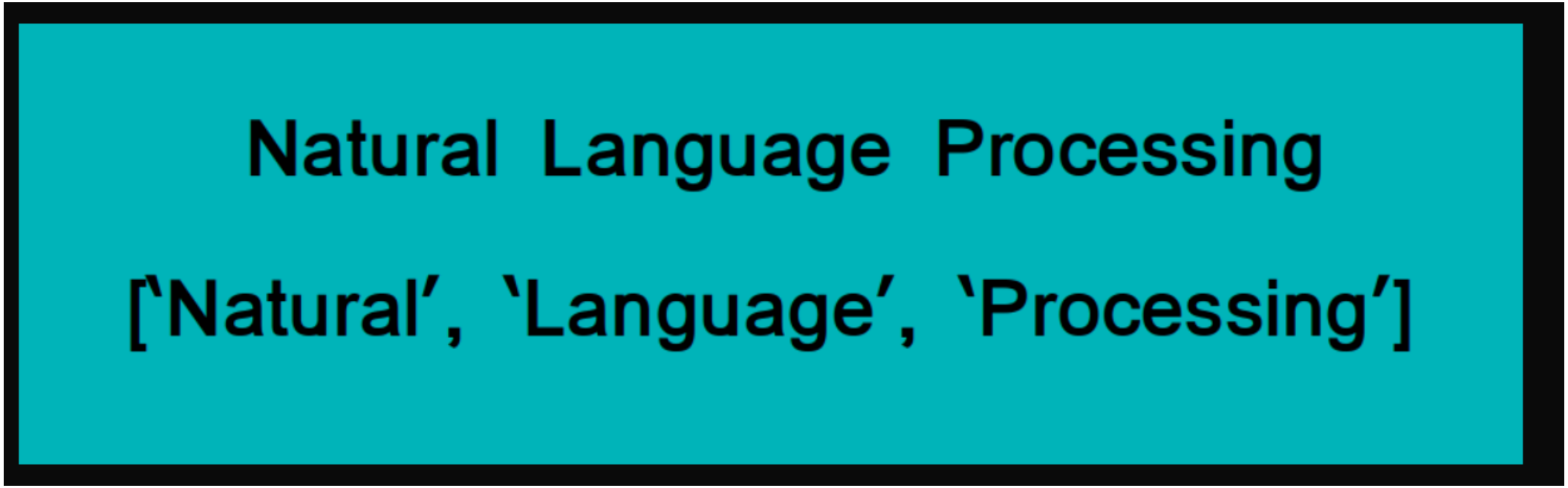
Tokenización por espacios

Separa el texto en los espacios. Cada fragmento resultante es un token.

- Es simple y rápida, pero puede dejar la puntuación junto a la palabra: `¡Hola,` | `mundo!`.
- También separa nombres de varias palabras, como `San` | `José`.

Aprendizaje profundo para PLN: ejemplo de tokenización

- La tokenización por espacios separa una oración en cada espacio.



Tokenización basada en diccionarios

Usa una lista de palabras conocidas para decidir dónde colocar los límites.

- Es útil en idiomas que no separan las palabras con espacios.
- Puede fallar ante palabras nuevas o varias segmentaciones posibles.

Ejemplo esquemático: `lacasaazul` se divide como `la | casa | azul` si esas palabras están en la lista.

Tokenización basada en reglas

Aplica patrones para decidir qué separar y qué conservar.

- Puede separar puntuación y reconocer números, fechas o abreviaturas.
- Las reglas dependen del idioma y de la tarea.

Ejemplo: `¡Hola, mundo!` puede separarse como `¡ | Hola | , | mundo | !`.

Convenciones y herramientas de tokenización

- **Penn Treebank** fija criterios para dividir casos como la puntuación y las contracciones.
- **spaCy** y **Moses** son herramientas que aplican reglas para generar tokens.

La convención fija los criterios. Las herramientas los aplican.

Tokenización por subpalabras: BPE y WordPiece

Si una palabra completa no está en el vocabulario, estos métodos pueden formarla con piezas conocidas.

BPE comienza con unidades pequeñas y combina las parejas vecinas más frecuentes del texto de entrenamiento.

Por ejemplo, `internacional` podría dividirse en `inter | nacional` si ambas piezas están en su vocabulario.

WordPiece aprende un vocabulario de piezas y busca la pieza disponible más larga. En algunos modelos, `##` marca la continuación de una palabra.

La división depende del vocabulario del modelo.

Tokenización por subpalabras: SentencePiece y unigramas

SentencePiece procesa el texto sin separar antes las palabras. Puede incluir los espacios en las piezas y marcarlos con `_`.

El modelo de unigramas asigna probabilidades a las piezas y elige la segmentación más probable. Durante el entrenamiento elimina las piezas que menos ayudan a explicar los datos.

SentencePiece es una herramienta. Puede usar unigramas o BPE como método de segmentación.

Aprendizaje profundo para PLN: representaciones numéricas

Las redes neuronales necesitan representaciones numéricas en lugar de palabras.

La **asignación de identificadores** asigna un número entero a cada palabra. Ese número es una etiqueta, no una medida de similitud.

I	like	writing	stories	on	the	computer.
1	2	3	4	5	6	7

La **codificación con un único valor activo** usa un vector con un 1 en la posición de la palabra y 0 en las demás. Representa identidad, no similitud.

	1	2	3	4	5	6	7
I	1	0	0	0	0	0	0
like	0	1	0	0	0	0	0
writing	0	0	1	0	0	0	0
stories	0	0	0	1	0	0	0
on	0	0	0	0	1	0	0
the	0	0	0	0	0	1	0
computer.	0	0	0	0	0	0	1

¿Por qué representar palabras mediante vectores?

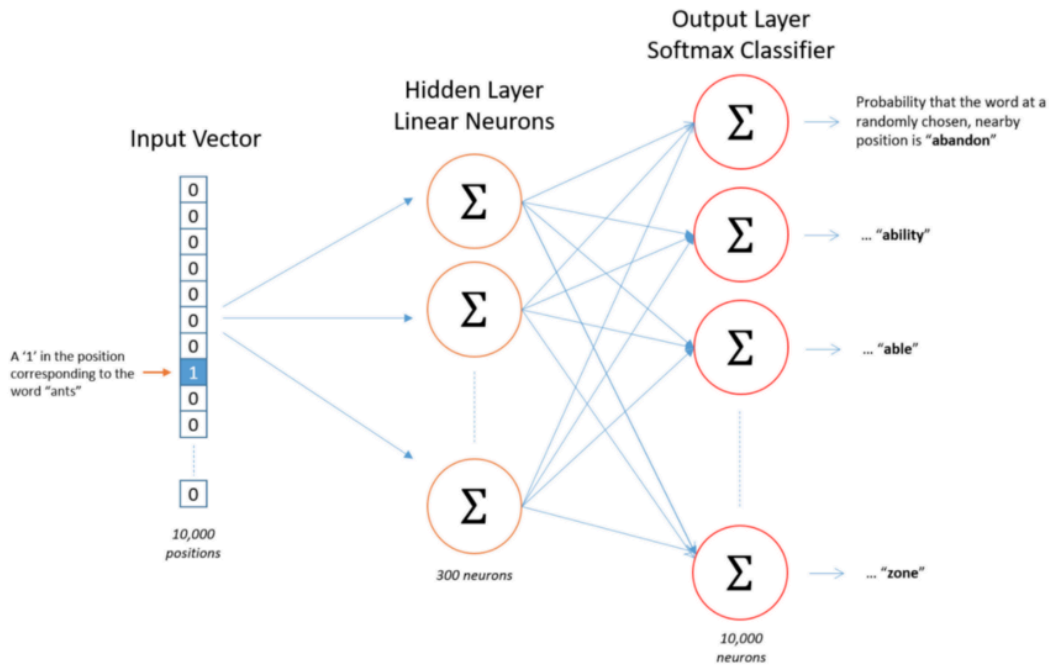
Asignar un identificador numérico a cada palabra permite distinguirla, pero no representa su significado.

Word2Vec aprende vectores a partir de los contextos en que aparecen las palabras.

- Palabras utilizadas en contextos similares tienden a tener vectores cercanos.
- La distancia entre vectores permite comparar sus representaciones.

Por ejemplo, *gato* y *perro* pueden tener vectores más similares que *gato* y *avión*.

Aprendizaje profundo para PLN: vectores de palabras



- La entrada con un único valor activo tiene 10 000 posiciones. El 1 resaltado señala la palabra de entrada.
- Una capa oculta con 300 neuronas produce su vector de representación. Σ indica la suma ponderada de una neurona.
- Una salida *softmax* estima una palabra cercana entre 10 000 opciones. Word2Vec aprende vectores mediante esta tarea de predicción.

Word2Vec: CBOW y Skip gram

Word2Vec aprende representaciones mediante dos tareas de predicción.

CBOW: utiliza las palabras vecinas para predecir la palabra central.

Skip gram: utiliza la palabra central para predecir las palabras vecinas.

Ejemplo: "El gato duerme sobre el sofá".

- CBOW: "El", "gato", "sobre", "el" → "duerme"
- Skip gram: "duerme" → "El", "gato", "sobre", "el"

En ambos casos, los vectores se ajustan durante el entrenamiento.

Entrenamiento de Word2Vec

1. Una ventana de contexto recorre el texto y genera pares de palabras centrales y vecinas.
2. En *Skip gram*, la palabra central predice sus vecinas. En *CBOW*, las vecinas predicen la palabra central.
3. Se ajustan los pesos para reducir el error de predicción. Los pesos aprendidos de la capa oculta se usan como vectores de palabras.

Las palabras que aparecen en contextos parecidos tienden a obtener vectores cercanos.

Propiedades y limitaciones de Word2Vec

Los vectores pueden representar relaciones entre palabras. Un ejemplo conocido es:

$$\vec{rey} - \vec{hombre} + \vec{mujer} \approx \vec{reina}$$

Estas relaciones no siempre se cumplen y dependen de los textos utilizados para entrenar el modelo.

Limitación principal: cada palabra tiene un vector fijo, independientemente de la oración.

- "Me senté en el banco de la plaza".
- "Deposité dinero en el banco".

Word2Vec utiliza el mismo vector para *banco* en ambas oraciones, aunque su significado sea diferente.

Aprendizaje profundo para PLN: contexto

Word2Vec asigna un vector fijo a cada palabra, pero su significado puede cambiar según el contexto.

Consideremos el ejemplo anterior:

- "Me senté en el **banco** de la plaza".
- "Deposité dinero en el **banco**".

Aunque la palabra es la misma, su significado es diferente.

Las representaciones contextuales consideran las demás palabras de la oración para representar cada término.

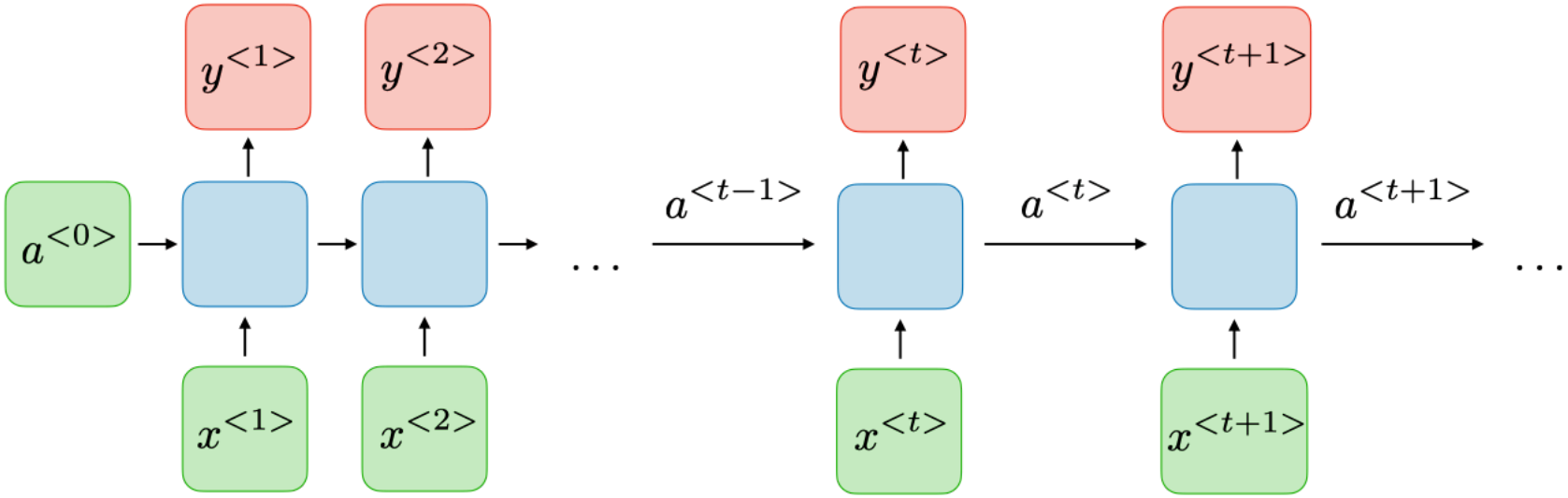
Las redes neuronales recurrentes (RNN) permiten comenzar a abordar este problema mediante el procesamiento de secuencias.

Aprendizaje profundo para PLN: redes recurrentes

- La RNN procesa la secuencia paso a paso.
- En cada paso, combina la entrada actual con el estado anterior.
- El estado recurrente conserva información de los elementos procesados.

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

x_t es la entrada actual y h_{t-1} el estado anterior. W_x y W_h son matrices aprendidas, b es un sesgo y f una función de activación.

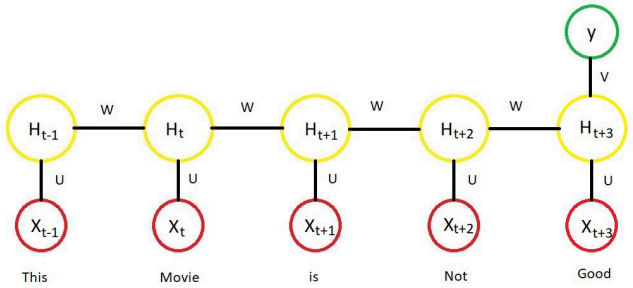


La red se despliega en el tiempo. Cada estado recibe una entrada y pasa información al siguiente. Fuente: Stanford CS230.

Aprendizaje profundo para PLN: uso de las RNN

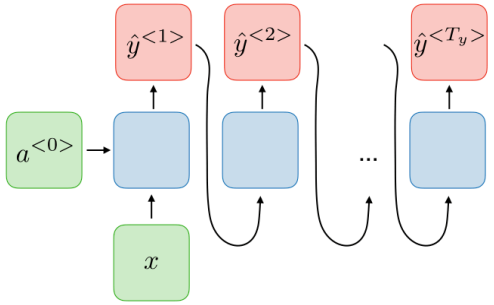
Clasificación de textos

La secuencia completa se resume en una etiqueta, como la categoría o el sentimiento.



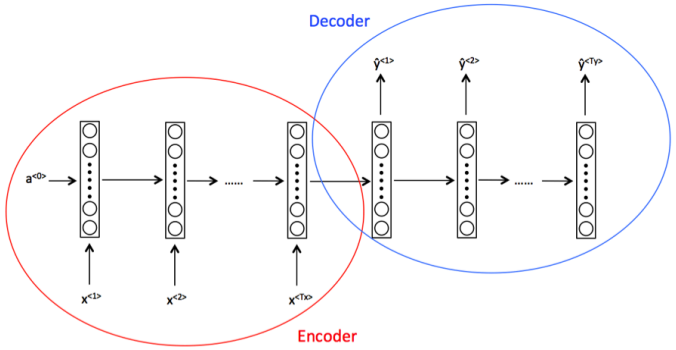
Predicción de palabras

En cada paso, el estado usa las palabras anteriores para estimar la siguiente.



Traducción automática

El codificador procesa la oración de entrada y el decodificador genera su traducción.



LSTM y GRU añaden compuertas que ayudan a conservar información relevante durante más pasos.

Diagramas de la presentación fuente. Fuente original: Analytics Vidhya.

Aprendizaje profundo para PLN: límites de las RNN

- En una RNN que avanza de izquierda a derecha, la información pasa de posiciones anteriores a posteriores, pero no vuelve desde el futuro.
- Puede ser difícil conservar dependencias lejanas en el estado recurrente.
- La atención ofrece una vía directa para usar información de posiciones relevantes de la secuencia de entrada.

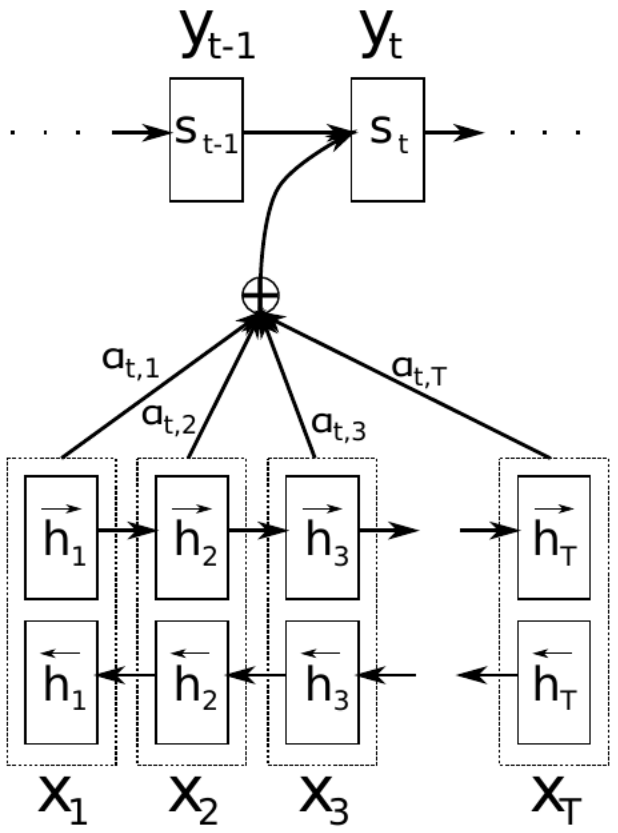
Aprendizaje profundo para PLN: RNN y atención

1. **Codificar.** Una RNN bidireccional procesa la entrada en ambos sentidos. Cada estado h_j combina información de las palabras anteriores y posteriores.
2. **Asignar pesos.** En cada paso t, el decodificador calcula pesos $\alpha_{t,j}$ que indican la relevancia de cada estado.
3. **Construir contexto.** Los estados se combinan mediante una suma ponderada:

$$c_t = \sum_{j=1}^{T} \alpha_{t,j} h_j$$

4. **Generar.** El decodificador utiliza c_t para ayudar a producir la siguiente palabra y_t.

Cada paso puede consultar distintas posiciones de la entrada.



Ejemplo de atención y secuencias largas

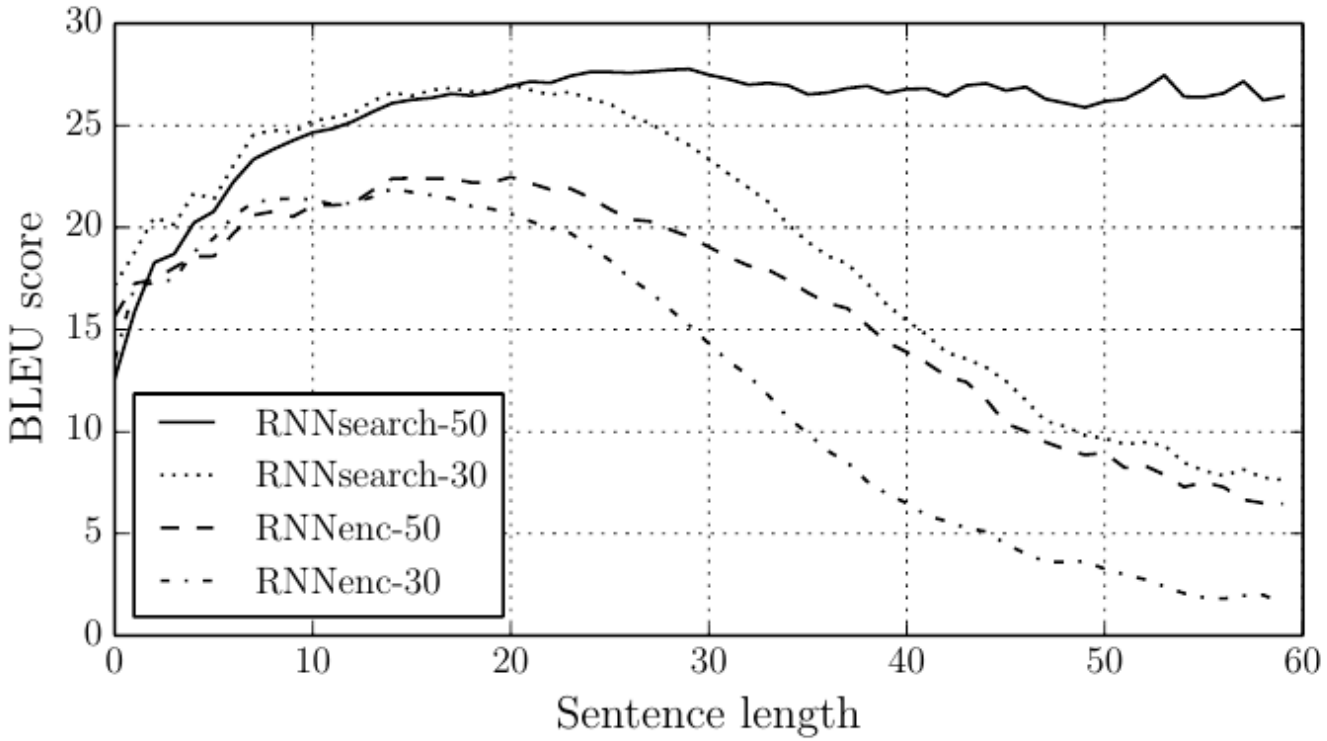
Se compara la calidad de traducción según la longitud de las oraciones, utilizando la métrica BLEU (Papineni et al., 2002). Un valor mayor indica una mejor traducción.

- **RNNenc:** modelo recurrente sin atención.
- **RNNsearch:** modelo recurrente con atención.

Resultado: ambos modelos pierden precisión en oraciones largas, pero el modelo con atención mantiene una calidad de traducción considerablemente mayor.

La atención permite consultar directamente distintas partes de la oración de entrada, reduciendo la dependencia de un único vector de contexto.

Atención y secuencias largas



De la atención a la autoatención

En el modelo anterior, el decodificador utiliza atención para consultar los estados del codificador.

La autoatención aplica el mismo principio dentro de una secuencia:

- Cada palabra compara su representación con las demás palabras, incluida ella misma.
- Estas comparaciones permiten calcular pesos de atención.
- Los pesos se utilizan para combinar información y obtener una nueva representación de cada palabra.

Así, cada posición incorpora información de otras posiciones de la misma secuencia.

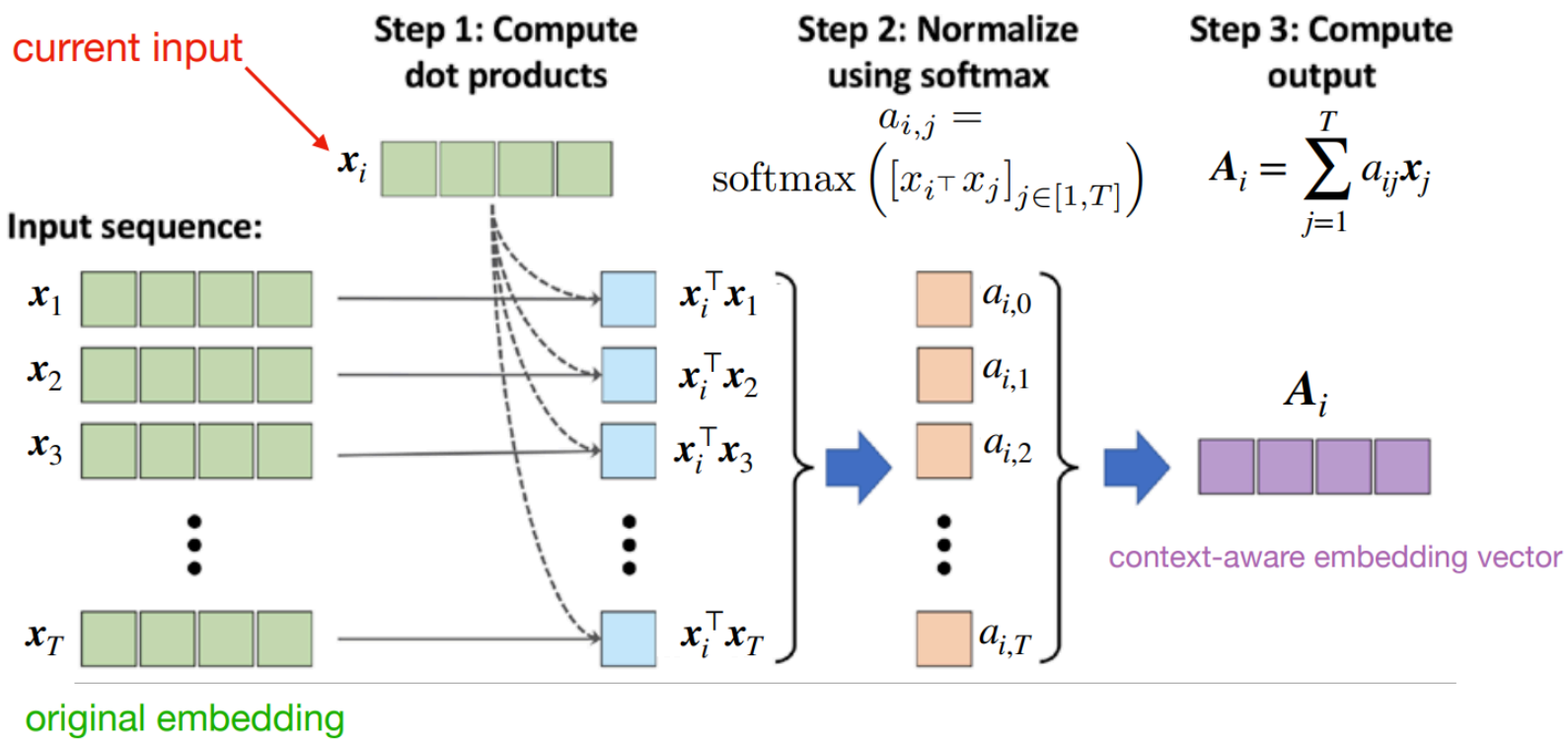
Aprendizaje profundo para PLN: autoatención

Una forma sencilla de implementar autoatención consiste en comparar los vectores de las distintas posiciones mediante el producto punto.

Para cada posición i:

1. Se calculan puntajes $x_i^\top x_j$ con todas las posiciones j.
2. Softmax transforma los puntajes en pesos a_{ij}.
3. Los pesos permiten combinar los vectores y obtener una nueva representación A_i.

Aprendizaje profundo para PLN: autoatención



Autoatención con parámetros aprendidos

En el ejemplo anterior, la autoatención compara directamente los vectores de entrada.

- Las operaciones son fijas y no contienen parámetros que puedan ajustarse durante el entrenamiento.
- Para aprender nuevas formas de comparar y combinar información, se introducen tres matrices de pesos.

$$q_j = W^q x_j, \qquad k_j = W^k x_j, \qquad v_j = W^v x_j$$

Estas matrices se ajustan durante el entrenamiento, permitiendo aprender qué características son relevantes para calcular la atención.

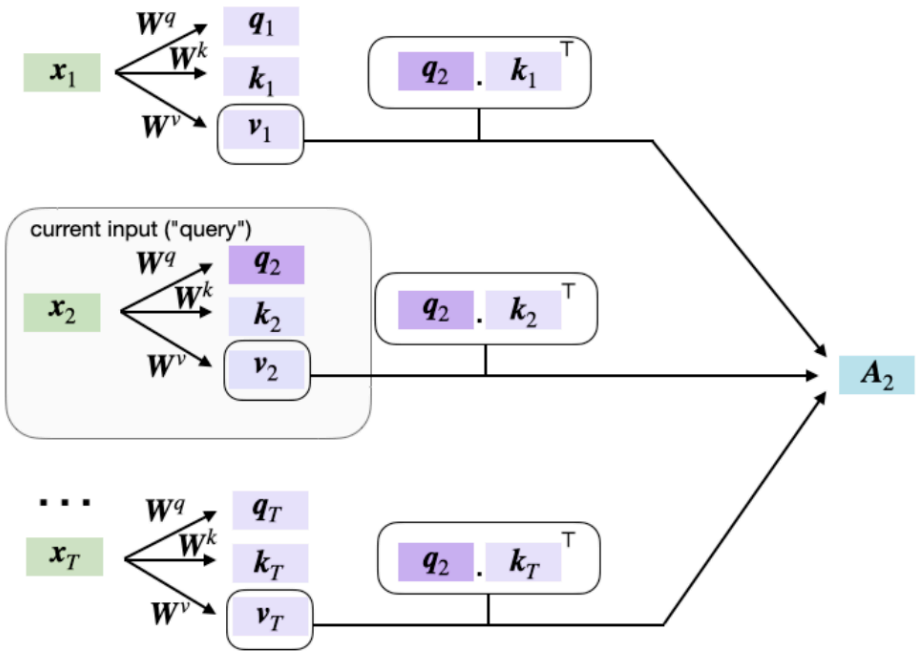
Aprendizaje profundo para PLN: atención de producto punto escalado

Cada vector de entrada x_j se transforma en tres representaciones:

- **Consulta (q_j):** representación de x_j que se utiliza para comparar esta posición con las demás y determinar cuáles son relevantes.
- **Clave (k_j):** representación de x_j con la que se comparan las consultas de otras posiciones para calcular los pesos de atención.
- **Valor (v_j):** representación de x_j que aporta información a la salida, ponderada según los pesos de atención calculados.

Las consultas se comparan con las claves mediante productos punto escalados. Los pesos resultantes se utilizan para combinar los valores.

Aprendizaje profundo para PLN: atención de producto punto escalado



¿Es necesaria la recurrencia?

Hasta ahora, hemos utilizado la atención para complementar el procesamiento secuencial de las RNN.

Sin embargo, la autoatención permite relacionar directamente distintas posiciones de una secuencia.

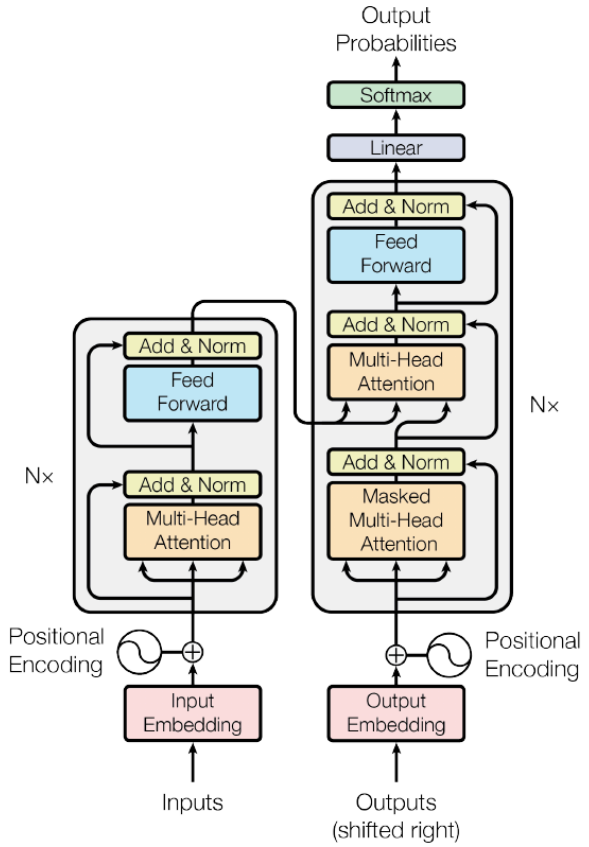
¿Podemos construir un modelo de lenguaje sin utilizar redes recurrentes?

En 2017, Vaswani et al. propusieron el **Transformer**, una arquitectura basada en mecanismos de atención que prescinde de la recurrencia.

Attention Is All You Need (Vaswani et al., 2017).

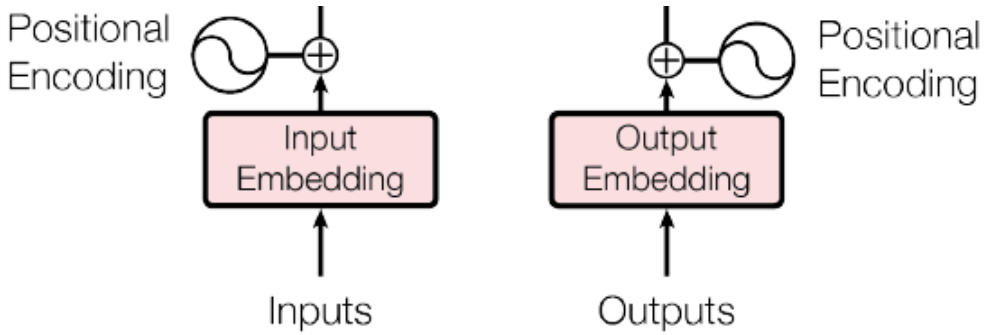
Aprendizaje profundo para PLN: arquitectura Transformer

- El codificador representa la entrada. El decodificador genera la salida paso a paso.
- Ambos apilan N capas de atención y redes prealimentadas.
- La codificación posicional añade el orden de los tokens.



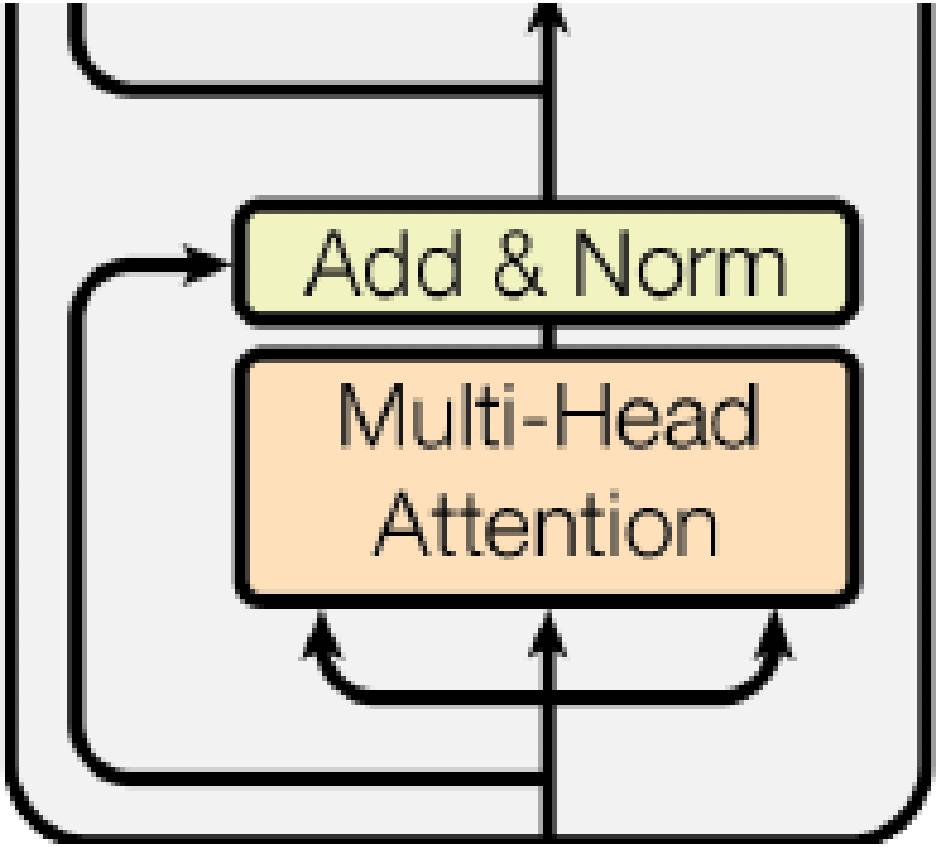
Vectores de entrada y posición

- Cada token se convierte en un vector de entrada.
- La codificación posicional se suma al vector para indicar el lugar del token en la secuencia.
- El codificador recibe la entrada. El decodificador recibe los tokens previos de la salida.



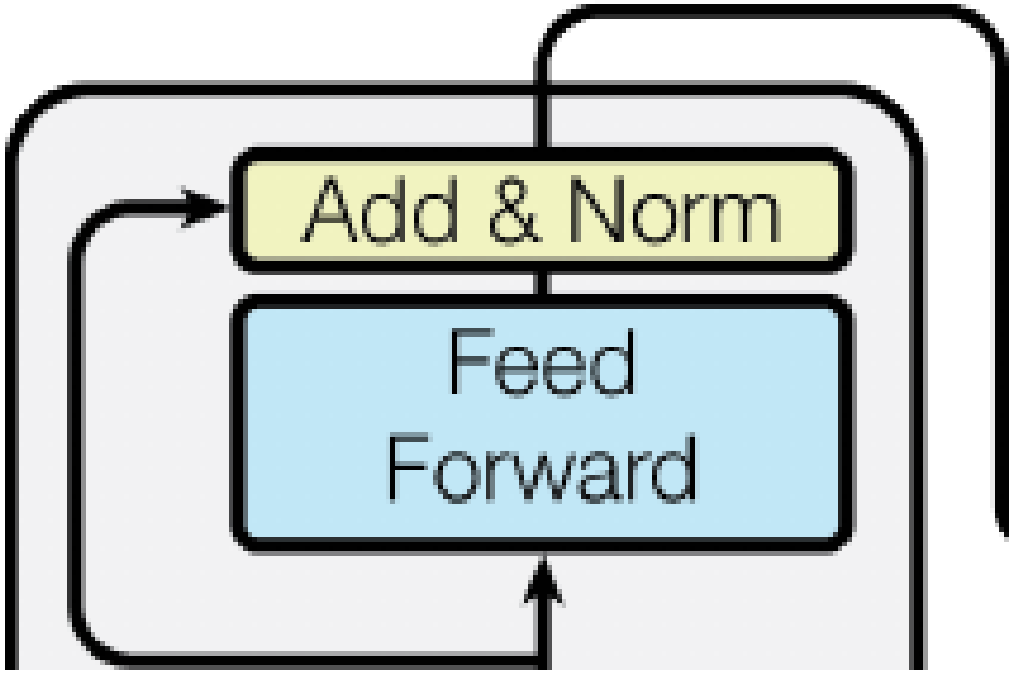
Atención del codificador

- Cada posición compara su consulta con las claves de la secuencia de entrada.
- Varias cabezas aprenden relaciones distintas entre posiciones.
- La salida de atención pasa a una conexión residual y normalización.



Conexión residual y normalización

- La conexión residual suma la entrada de la subcapa a su salida.
- Después, la normalización ajusta el resultado.
- La ruta directa ayuda a conservar información y a entrenar redes profundas.



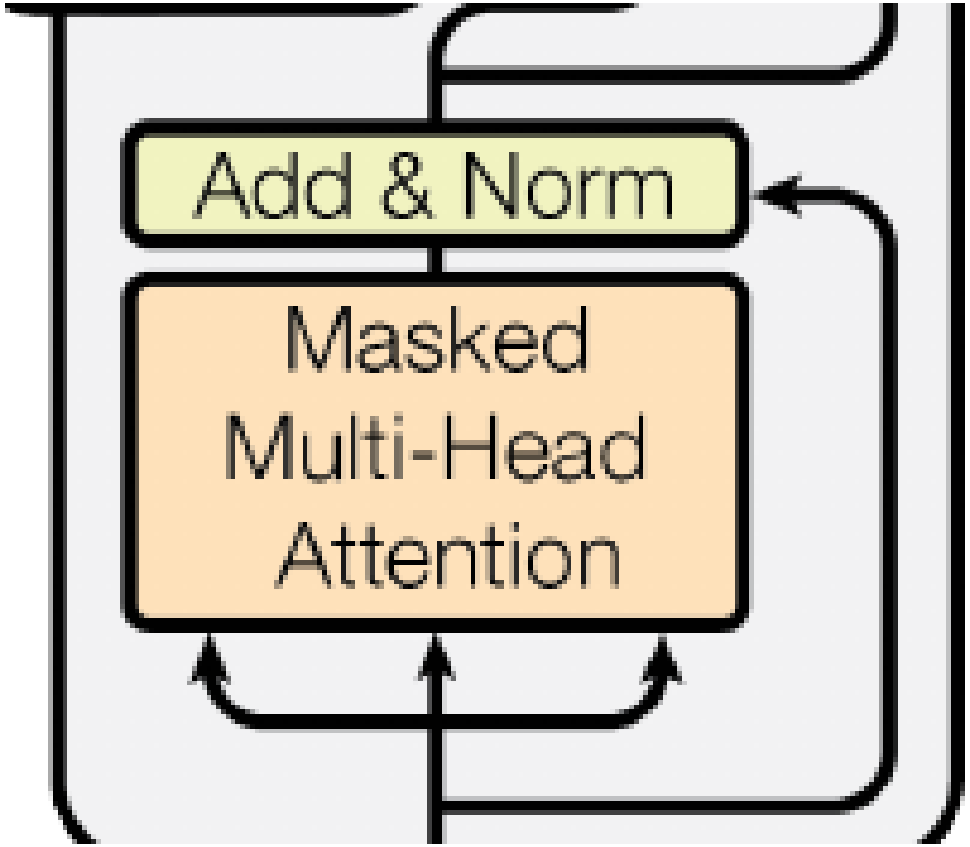
Red prealimentada por posición



- La red transforma cada posición por separado.
- Usa la misma transformación aprendida en todas las posiciones.
- A diferencia de la atención, no mezcla información entre tokens.

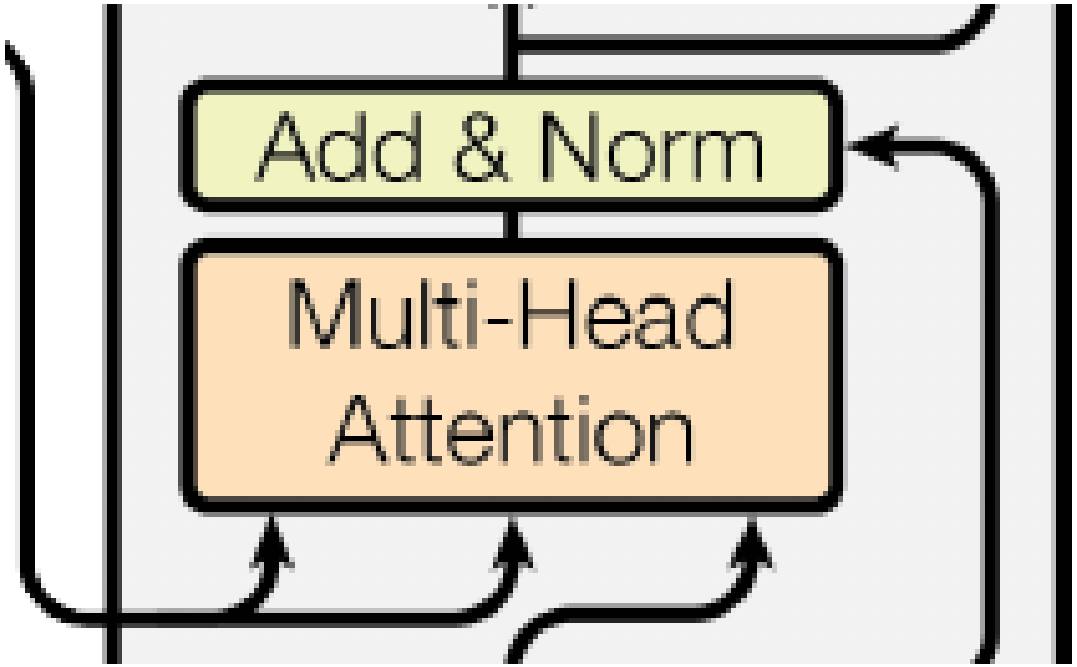
Atención enmascarada del decodificador

- Durante la generación, una posición solo puede usar tokens anteriores y el actual.
- La máscara oculta las posiciones futuras para que el modelo no anticipe la respuesta.



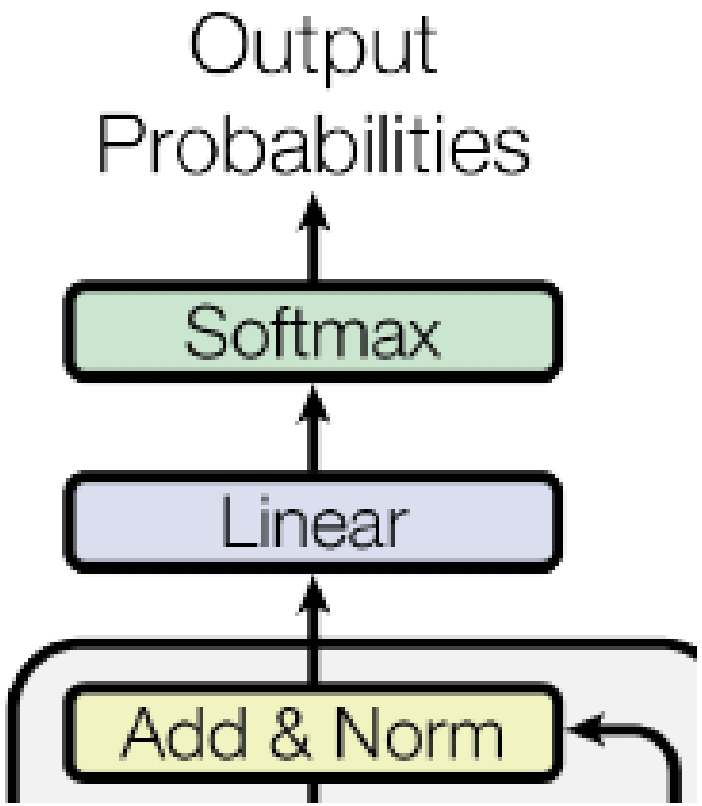
Atención entre codificador y decodificador

- Las consultas vienen del decodificador.
- Las claves y los valores vienen del codificador.
- Así, cada palabra generada puede recuperar las partes relevantes de la entrada.



Proyección final y probabilidades

- La proyección lineal calcula un puntaje para cada token del vocabulario.
- *Softmax* convierte los puntajes en probabilidades.
- Estas probabilidades permiten predecir el siguiente token.



Usos actuales del procesamiento del lenguaje natural

- **Asistentes multimodales:** ChatGPT, Gemini y Claude comprenden y generan texto, imágenes o audio.
- **Programación:** GitHub Copilot y Cursor ayudan a escribir, explicar y revisar código.
- **Traducción:** DeepL traduce textos y facilita la comunicación entre idiomas.
- **Búsqueda y respuestas:** Perplexity combina búsqueda web con respuestas sintetizadas y fuentes.
- **Documentos y empresas:** sistemas con generación aumentada por recuperación responden usando manuales y bases de conocimiento propias.
- **Voz y accesibilidad:** transcripción, subtítulos y asistentes de voz convierten habla en texto y texto en habla.

Estas aplicaciones se integran en educación, investigación, atención al cliente y desarrollo de software.

Detrás de estas aplicaciones, la atención selecciona qué partes del contexto contribuyen a cada salida. Veamos cómo lo hace.

¿Cómo construye contexto la atención?

Para enriquecer la representación de una posición, el modelo compara lo que necesita con la información disponible en las demás posiciones.

Símbolo	Significado
q_i	Consulta: qué información busca la posición i
k_j	Clave: con qué compara la consulta en la posición j
v_j	Valor: qué contenido aporta la posición j
a_{ij}	Peso que la consulta q_i asigna a la posición j
o_i	Representación contextual de la posición i

Las matrices Q, K, V reúnen las consultas, claves y valores de toda la secuencia. En autoatención se calculan desde la misma secuencia. En atención cruzada, Q viene del decodificador y K, V del codificador, como en el diagrama del Transformer anterior.

De la entrada a Q, K y V

Sea X la secuencia de entrada con n posiciones y d_{model} características:

$$X \in \mathbb{R}^{n \times d_{\text{model}}}$$

Tres proyecciones aprendidas convierten cada representación de entrada en consulta, clave y valor:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

Si $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ y $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, entonces:

$$Q, K \in \mathbb{R}^{n \times d_k}, \qquad V \in \mathbb{R}^{n \times d_v}$$

W_Q, W_K, W_V se aprenden durante el entrenamiento. d_k es la dimensión de consultas y claves, y d_v la dimensión de los valores.

Ejemplo de dimensiones

Considere cuatro posiciones, con $d_{\text{model}} = 6$, $d_k = 3$ y $d_v = 2$:

$$X \in \mathbb{R}^{4\times 6}, \qquad Q, K \in \mathbb{R}^{4\times 3}, \qquad V \in \mathbb{R}^{4\times 2}$$

El producto $QK^\top$ compara cada consulta con cada clave y genera un puntaje por pareja:

$$QK^\top : \ (4 \times 3)(3 \times 4) = 4 \times 4$$

La matriz de pesos A también es 4×4. Al multiplicarla por V, se obtiene una representación contextual por posición:

$$AV : \ (4 \times 4)(4 \times 2) = 4 \times 2$$

De puntajes a salida contextual

1. Puntaje. La consulta q_i se compara con cada clave k_j. d_k es la dimensión de esos vectores:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

2. Peso. Softmax normaliza los puntajes sobre las posiciones j. Así, los pesos de cada consulta suman uno:

$$a_{ij} = \mathrm{softmax}_j(s_{ij}), \qquad \sum_{j=1}^{n} a_{ij} = 1$$

3. Salida. Los pesos combinan los valores para formar la representación de la posición i:

$$o_i = \sum_{j=1}^{n} a_{ij} v_j$$

Forma matricial de la atención

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Cada fila de $QK^\top$ contiene puntajes entre una consulta y todas las claves. $\top$ indica la transpuesta de K. Dividir por $\sqrt{d_k}$ evita puntajes demasiado grandes antes de softmax. Softmax por fila produce pesos que suman uno. AV usa esos pesos para combinar V.

Ejemplo numérico de puntajes

Considere dos posiciones y dimensión $d_k = 2$:

$$q_1 = (1, 0), \qquad k_1 = (1, 0), \qquad k_2 = (0, 1)$$

El producto punto mide alineación: q_1 coincide con k_1 y es ortogonal a k_2.

$$q_1 K^\top = (1, 0)$$

Al escalar por $\sqrt{2}$:

$$\left(\frac{1}{\sqrt{2}}, 0 \right) \approx (0.7071, 0)$$

El escalamiento reduce la magnitud de los puntajes antes de aplicar softmax. A continuación, esos puntajes se convierten en pesos de atención.

Ejemplo numérico de pesos y salida

Softmax convierte los puntajes en pesos positivos que suman uno. En el orden de las claves k_1, k_2, los pesos a_{11}, a_{12} son aproximadamente 67% y 33%:

$$\mathrm{softmax}(0.7071, 0) \approx (0.6698, 0.3302)$$

Esos pesos se aplican a los valores v_1, v_2. La salida contextual o_1 para la primera consulta es:

$$o_1 = 0.6698 v_1 + 0.3302 v_2 = (0.6698, 0.3302)$$

La salida queda entre v_1 y v_2, con mayor contribución de v_1 porque su peso es mayor.

Autoatención en el Transformer

En el ejemplo anterior se calculó una salida para una consulta. En autoatención, cada posición de la secuencia genera una consulta y recibe contexto. Q, K y V se calculan a partir de la misma entrada X.

El esquema introductorio comparaba directamente los vectores de representación. El Transformer aprende proyecciones para ajustar las comparaciones durante el entrenamiento.

Atención de varias cabezas

Una cabeza aprende una proyección y calcula una salida contextual. Para representar distintas relaciones, el Transformer usa h cabezas. i identifica una cabeza y W_i^Q, W_i^K, W_i^V son sus matrices aprendidas:

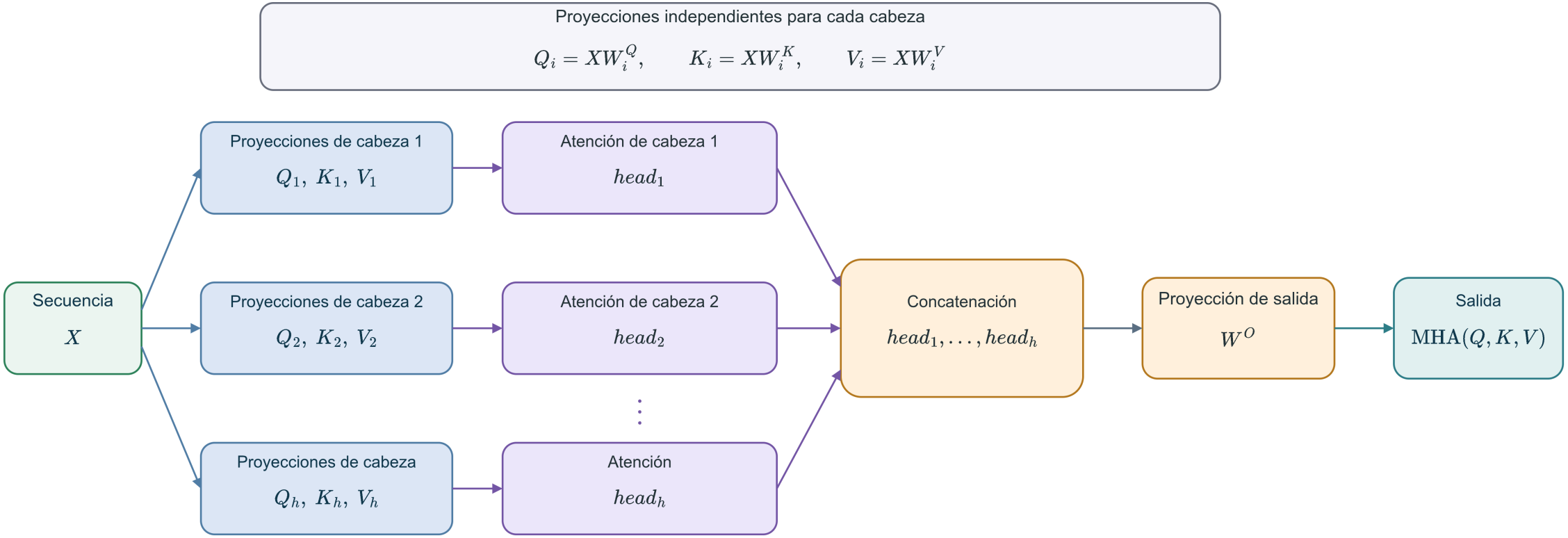
$$head_i = \text{Atención}(QW_i^Q, KW_i^K, VW_i^V)$$

Las salidas se concatenan, no se promedian:

$$\text{MHA}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O$$

MHA significa atención de varias cabezas. W^O mezcla las salidas concatenadas y las proyecta a la dimensión del modelo.

Flujo de atención de varias cabezas



La entrada X se proyecta en paralelo para las h cabezas. Sus salidas se concatenan antes de aplicar W^O.

Dimensiones de varias cabezas

En la configuración original, h es el número de cabezas y las dimensiones de cada una se calculan así:

$$d_k = d_v = \frac{d_{\text{model}}}{h}$$

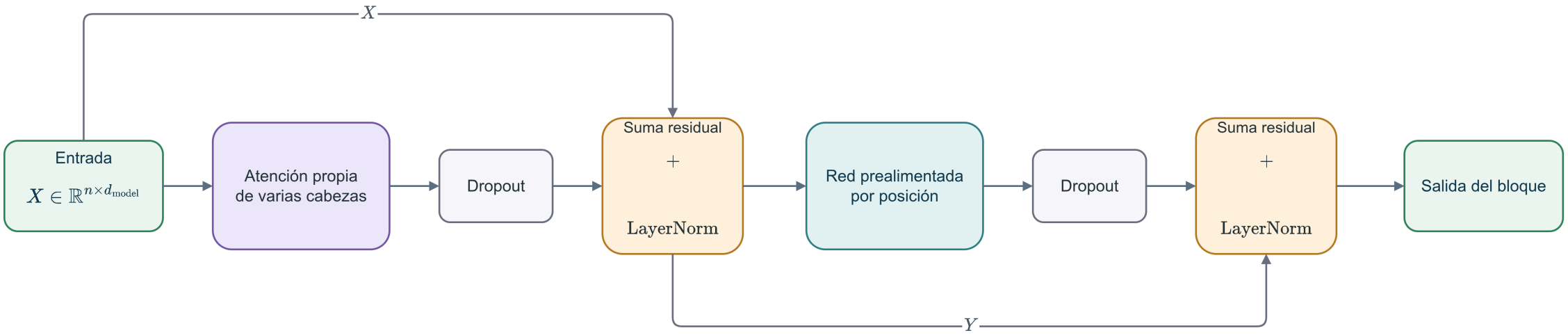
Por ejemplo, con $d_{\text{model}} = 512$ y $h = 8$, cada cabeza trabaja con dimensión 64. Al concatenar las ocho salidas se recuperan $8 \cdot 64 = 512$ características antes de aplicar la matriz de salida W^O.

Las dimensiones deben ser compatibles. Otras variantes pueden distribuir consultas, claves y valores de otra manera.

De la atención al bloque Transformer

Ya vimos cómo una cabeza calcula el contexto y cómo varias cabezas reúnen relaciones distintas. En el codificador, la atención se combina con conexiones residuales, normalización y una red prealimentada.

Bloque original del codificador



$$Y = \text{LayerNorm}\left(X + \text{Dropout}(\text{MHA}(X))\right)$$

$$Z = \text{LayerNorm}\left(Y + \text{Dropout}(\text{FFN}(Y))\right)$$

Fuente: Vaswani et al. (2017). X es la entrada (n posiciones y d_{model} características). Y es la salida de MHA y Z la de FFN. MHA es atención de varias cabezas y FFN una red prealimentada. La suma residual combina la entrada y la salida de cada subcapa. La normalización por capa se aplica después. La desactivación aleatoria de neuronas ayuda a reducir el sobreajuste durante el entrenamiento.

Conexiones residuales y normalización

El diagrama muestra dos subcapas: atención de varias cabezas (MHA) y red prealimentada (FFN). En la versión original, cada una se combina con su entrada residual y luego se normaliza:

$$Y = \mathrm{LayerNorm}(X + \mathrm{Dropout}(\mathrm{MHA}(X)))$$

$$Z = \mathrm{LayerNorm}(Y + \mathrm{Dropout}(\mathrm{FFN}(Y)))$$

Y es la salida tras atención y Z tras FFN. La desactivación aleatoria de neuronas se usa durante el entrenamiento. La normalización por capa normaliza las características de cada posición. Cada suma residual usa la entrada de su propia subcapa.

Red prealimentada por posición

La atención mezcla información entre posiciones. Después, la red prealimentada aplica la misma transformación a cada posición por separado:

$$\mathrm{FFN}(x) = W_2\,\mathrm{ReLU}(W_1 x + b_1) + b_2$$

En la ecuación, x es el vector de una posición, W_1, W_2 son matrices aprendidas y b_1, b_2 son sesgos. ReLU (unidad lineal rectificada) conserva los componentes positivos y sustituye por cero los negativos. FFN comparte sus parámetros entre posiciones. Así, la atención mezcla información entre posiciones y FFN transforma cada una.

Máscara para relleno

Para procesar juntas secuencias de distinta longitud, se agregan tokens de relleno y se forman lotes. Como esos tokens no contienen información, la atención debe ignorarlos.

- Antes de softmax, se enmascaran las posiciones de relleno usadas como claves.
- Así, una consulta real no asigna atención a contenido artificial.
- Las salidas de las posiciones de relleno se ignoran al calcular la pérdida.

La máscara de relleno identifica posiciones inválidas. No impide que dos posiciones reales se atiendan entre sí.

Máscara causal



En generación, el decodificador solo puede usar tokens anteriores y el actual. La máscara causal bloquea los futuros. Si además hay relleno, ambas máscaras se pueden combinar.

Codificación posicional sinusoidal

Las máscaras definen qué posiciones pueden interactuar, pero no indican su orden. La codificación posicional aporta esa información. En el Transformer original se suma al vector de representación de cada token. pos es el índice del token e i identifica el par de dimensiones par $2i$ e impar $2i+1$:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Las dimensiones usan frecuencias distintas: las rápidas cambian más entre posiciones cercanas y las lentas varían gradualmente. El número 10000 fija la escala de esas frecuencias. También existen vectores posicionales aprendidos.

Posición relativa

Una alternativa a sumar una posición absoluta es ajustar el puntaje de atención según la distancia entre dos posiciones:

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + b_{i-j}$$

Aquí q_i es la consulta de la posición i, k_j la clave de la posición j y d_k su dimensión. El sesgo aprendido b_{i-j} depende del desplazamiento entre ambas posiciones. Así, el mismo desplazamiento recibe el mismo ajuste aunque cambie la ubicación absoluta de los tokens.

Codificación rotatoria de posiciones (RoPE)

RoPE incorpora la posición al rotar pares de características de las consultas y las claves:

$$q'_m = R_m q_m, \qquad k'_n = R_n k_n$$

Aquí R_m es la matriz de rotación para la posición m y la prima indica que el vector fue rotado. m y n son índices de posición. En el producto punto, R_{n-m} es la rotación correspondiente a su desplazamiento relativo:

$$(q'_m)^\top k'_n = q_m^\top R_{n-m} k_n$$

RoPE se aplica a Q y K. No se suma como un vector posicional a las representaciones de entrada.

Comparación de métodos posicionales

Método	Dónde se incorpora	Qué aporta
Sinusoidal o aprendida	A la entrada, sumada al vector de representación	Posición absoluta de cada token
Sesgo relativo	Al puntaje, mediante b_{i-j}	Distancia entre consulta y clave
RoPE	Rotando q_i y k_j antes del producto	El puntaje depende del desplazamiento relativo

En la tabla, i identifica la consulta y j la clave. La diferencia práctica es dónde entra la posición: en la representación de entrada o en el cálculo de atención.

Cómo se organizan codificador y decodificador

Las máscaras y la información posicional permiten distintos patrones, según cómo se lee la entrada y cómo se produce la salida:

Patrón	Qué hace	Ejemplo
Codificador	Representa la entrada con contexto bidireccional	BERT
Solo decodificador	Predice el siguiente token con atención causal	GPT
Codificador y decodificador	Codifica una entrada y genera una salida con atención cruzada	T5

BERT usa un codificador bidireccional. GPT genera texto de forma autoregresiva. T5 representa distintas tareas como transformaciones de texto a texto. En T5, la atención cruzada permite al decodificador consultar la entrada. El preentrenamiento y la adaptación se estudian en 05-3.

Costo de atención estándar

La matriz de atención calcula un puntaje por pareja de consulta y clave. Por eso, para una secuencia de longitud n, tiene forma $n \times n$:

$$\text{memoria de puntajes} = O(n^2), \qquad \text{cálculo} = O(n^2 d)$$

La notación $O(\cdot)$ describe cómo crece el costo, no el tiempo exacto. Almacenar los puntajes requiere memoria $O(n^2)$ y calcular la atención densa cuesta aproximadamente $O(n^2 d)$, donde n es la longitud y d el ancho de las características.

Caché de claves y valores (KV)

En la generación autoregresiva se produce un token nuevo por paso. Las claves y valores del prefijo no cambian, así que la caché KV los guarda para reutilizarlos en cada capa.

- Sin caché, al extender el prefijo se volverían a calcular las claves y los valores de los tokens anteriores.
- Con caché, solo se añaden las claves y los valores del nuevo token.
- La nueva consulta aún debe compararse con las claves almacenadas. La caché evita repetir cálculos, pero ocupa más memoria a medida que crece el contexto.

Reducir la memoria de la caché

- En MHA, cada cabeza de consulta tiene sus propias claves y valores.
- MQA comparte una cabeza de claves y valores entre todas las consultas.
- GQA comparte las claves y los valores dentro de grupos de consultas.

K/V abrevia claves y valores. Con ocho cabezas de consulta, MHA puede guardar ocho pares, GQA dos y MQA uno. El número de consultas no cambia. Compartir claves y valores reduce el espacio y el tráfico de memoria de la caché.

Atención optimizada por bloques (FlashAttention)

FlashAttention calcula la misma atención exacta que se explicó antes, pero por bloques para aprovechar mejor la memoria rápida del acelerador.

- Reduce lecturas y escrituras de memoria.
- Evita materializar la matriz completa de puntajes en memoria de alto ancho de banda.
- Mantiene el costo aritmético cuadrático de la atención densa.

El cálculo por bloques conserva el resultado exacto de la atención densa.

Mezcla de expertos (MoE)

La mezcla de expertos es otra forma de reducir el cálculo, esta vez en la red prealimentada del bloque Transformer. Divide esa red en expertos y un enrutador selecciona cuáles procesan cada token.

- Los parámetros totales pueden crecer sin activar todos los expertos en cada paso.
- Los expertos no seleccionados no procesan ese token. El enrutamiento debe equilibrar la carga para que no se concentre en pocos expertos.
- Es una técnica de cómputo condicional, no un mecanismo de atención.

Qué optimiza cada mecanismo

Mecanismo	Qué ahorra o mejora	Coste o límite
Caché KV	Recalcular K y V del prefijo en cada paso	La memoria crece con el contexto
MQA y GQA	Tamaño y tráfico de la caché de K y V	La atención sigue comparando con el contexto
FlashAttention	Lecturas y escrituras de memoria del acelerador	El cálculo denso sigue siendo cuadrático
Mezcla de expertos	Cómputo activo por token frente a parámetros totales	Enrutamiento y equilibrio de carga

Cada método atiende un costo distinto: la caché evita repetir proyecciones, MQA y GQA reducen su tamaño, FlashAttention reduce el movimiento de datos y la mezcla de expertos limita el cálculo activo en la red prealimentada.

Síntesis de la unidad

- La atención combina información entre posiciones mediante Q, K y V.
- Las máscaras y la posición controlan qué relaciones se representan.
- El Transformer compone atención, conexiones residuales, normalización y redes prealimentadas.
- Las variantes modernas abordan memoria, velocidad de inferencia y cómputo condicional.

La unidad 05-3 estudia modelos de lenguaje y su adaptación. La generación aumentada por recuperación (RAG) y su evaluación se reservan para 05-4.

El cuaderno práctico con PyTorch permite repetir el cálculo numérico y explorar las máscaras y la atención de varias cabezas.

Referencias

- Aprendizaje profundo en detalle, capítulo 11
- El Transformer comentado, Harvard NLP
- Vaswani et al., Atención es todo lo que necesitas, 2017
- Su et al., RoFormer, 2021
- Raschka, apuntes sobre RNN y Transformers, sesión 19
- Raschka, S. y Mirjalili, V. (2019). *Aprendizaje automático con Python*, 3.ª edición.
- Tokenización para PLN, Towards Data Science
- Tokenización por espacios, Analytics Vidhya
- Ainslie et al., GQA, 2023
- Dao et al., atención Flash, 2022
- Shazeer et al., mezcla de expertos, 2017
- Stanford CS336, modelado de lenguaje desde cero
- Bahdanau, Cho y Bengio, traducción automática neuronal con alineación aprendida conjuntamente (2014)
- Vectores de palabras, codificación con un único valor activo y salto de gram, Towards Data Science
- Proyector de TensorFlow
- Asistentes conversacionales: ChatGPT, Gemini y Claude.
- Otras herramientas: GitHub Copilot, Cursor, DeepL y Perplexity.